

CPSC 335, Algorithm Engineering – Spring 2026

Project 2 Part B: Greedy versus Exhaustive

Continuing from a completed Part A

What to Do

1. Project Implementation:

- a. Implement, test, and deliver your algorithms' implementation
- b. Run your program several times using different scenario files and saving each run's output to a unique text file
 - i. You are provided several Scenarios-*.txt files. Each file has 1 or more scenarios. The parameters of a scenario are the weight limit you can carry, the minimum and maximum weight of food items used to form your food item selection pool, and the size of your selection pool. These are just starting points. You are to create your own additional scenarios to validate and test your solutions.
 - ii. Redirect standard input from a scenario file and redirect standard output to an output text file.
- c. When everything is working well, run your program using Scenarios-ForDelivery.txt as input and creating output-ForDelivery.txt with your output. Turn in output-ForDelivery.txt with your submission.

2. Project Analysis and Conclusions

- a. Produce and deliver brief handwritten Analysis Report scanned into PDF format. Your handwriting must be absolutely and perfectly legible to earn credit. Treat this not as work in progress, but rather your final, polished, college-level quality delivery to your customer. Your Analysis Report should include responses to the following:
 - i. Draw a scatter plot for each algorithm and fit line for the data generated using Scenarios-ForDelivery.txt as input. The instance (a.k.a. selection pool) size n should be on the horizontal axis and elapsed time should be on the vertical axis. Each plot should have a title and each axis should have a label and units of measure.
 - ii. A brief conclusion based solely on your collected and plotted data above indicating whether or not your empirically observed time efficiency data is consistent, or inconsistent, with your Part A's mathematically derived big-O efficiency class for each algorithm. Justify your answer.
 - iii. Is there a noticeable difference in the performance of the two algorithms? Which is faster, and by how much? Does this surprise you?
 - iv. Is this evidence consistent or inconsistent with Part A's hypothesis 1? Justify your answer.
 - v. Is this evidence consistent or inconsistent with Part A's hypothesis 2? Justify your answer.

Rules and Constraints:

1. You are to modify only designated TO-DO sections using only the techniques discussed in class. **The grading process will detect and discard any changes made outside the designated TO-DO sections, including spacing and formatting.** Designated TO-DO sections are identified with the following comments:

```

//////////////////// TO-DO (X) //////////////////////
...
//////////////////// END-TO-DO (X) //////////////////////

```

Keep and do not alter these comments. Insert your code between them.

2. You may work in groups of up to 2 people, but you must deliver individually. The Analysis Report must be individually handwritten by each group member (No, you can't deliver your groupmates' handwritten report as your own.)
3. Multiple submissions may be available and a waiting period between submissions may be enforced.

Reminders:

- The C++ using directive `using namespace std;` is **never allowed** in any file in any deliverable product. Being new to C++, you may have used this in the past. If you haven't done so already, it's now time to shed this crutch and fully decorate your identifiers.
- A clean compile is an entrance criterion. Deliveries that do meet the entrance criteria cannot be graded.
- The grading process uses Build.sh on Ubuntu to compile and link your program. There is nothing magic about Build.sh, all it does is save you (and me) from repeatedly typing the very long compile command and all the source files to compile.
- Filenames are case sensitive, both in source code and in your OS file system. Windows doesn't care about filename case, but Linux does.
- You may redirect standard input from a text file, and you must redirect standard output to a text file named output-ForDelivery.txt. Failure to include output-ForDelivery.txt in your delivery indicates you were not able to execute your program and will be scored accordingly. A screenshot of your terminal window is not acceptable. See [How to build and run your programs](#). Also see [How to use command redirection under Linux](#) if you are unfamiliar with command line redirection.

Deliverable Artifacts:

Provided files	Files to deliver	Comments
Algorithm.hpp	1. Algorithm.hpp	Start with the file provided. Make changes in the designated TO-DO sections (only). The grading process will detect and discard all other changes.
FoodItem.hpp FoodPantry.hpp main_multithreaded.cpp Timer.hpp	2. FoodItem.hpp 3. FoodPantry.hpp 4. main_multithreaded.cpp 5. Timer.hpp	You shall not modify these files. The grading process will overwrite whatever you deliver with the one provided with this assignment. It is important your delivery is complete, so don't omit these files.
	6. output-ForDelivery.txt	Capture your program's output to this text file using command line redirection. See command redirection . Failure to deliver this file indicates you could not get your program to execute. Screenshots or terminal window log files are not permitted.
Report_Template.pdf	7. Project_2B_Report.pdf	Print the template, add your name, CWID, and handwritten report content. Add additional pages if necessary. Scan your completed work and save as a .pdf file named as specified in the previous column.
	Readme.txt	Optional. Use it to communicate your thoughts to the grader.
Food.csv		Comma separated value (csv) text file containing thousands of food items. Each item has a positive integral weight in ounces and a non-negative floating point calorie rating.
Scenarios-ForDelivery.txt Scenarios-ForDevelopment.txt Scenarios-ForTesting.txt		Text files to be used as input to your program containing one or more scenarios. The grading process may use a different scenario file.
sample_output.txt		A sample of a working program's output. Your output may vary.